



Experiment – 1

Name: Rameshwor Chaudhary

UID: 24BAI71012

Branch: CSE(AIML)-CSS21

Section/Group: 24AIT-NTTP-3[B]

Semester: 5th

Date of Performance: 08/07/2026

Subject: Full Stack-II

Subject Code: 24CSP-337

EXPERIMENT – 1.1

1. **AIM:** To design and develop a dynamic post composer interface supporting multiple platforms with constraint validation.

2. **OBJECTIVE:**

1. To gain practical knowledge of the given concept.
2. To develop logical thinking and problem-solving skills.
3. To implement the program correctly and analyze its output.
4. To improve coding efficiency and debugging skills.

3. **SOFTWARE REQUIREMENTS:**

1. **Operating System:** Windows 10/11 or Linux
2. **Programming Software/IDE:** Visual Studio Code /MySQL Workbench
3. **Processor:** Intel Core i3 or higher
4. **RAM:** Minimum 4 GB

4. **THEORY**

This experiment is designed to provide a practical understanding of the given concept through implementation. It focuses on applying theoretical knowledge to solve real-world programming problems. By writing, compiling, and executing the program, students can observe how the logic works and how the expected output is generated. Practical experiments help strengthen programming fundamentals and improve analytical and debugging skills.

In this experiment, React.js is used to build the user interface because it supports component-based development and dynamic rendering. The useState Hook is used to manage the application's state, including the post content, selected platform, and saved drafts. Whenever the user enters text or selects a different platform, React updates the interface automatically without reloading the page. React's state-driven approach helps create responsive and interactive user experiences.

5. IMPLEMENTATION/PROCEDURE

1. Created a React application using Create React App.
2. Imported the useState Hook to manage the application state.
3. Created state variables for Post, Platform, and Drafts.
4. Designed the user interface with a dropdown menu, text area, and Save Draft button.
5. Added multiple social media platform options such as Instagram, Facebook, LinkedIn, and X.
6. Implemented input validation to ensure that empty posts cannot be saved.
7. Stored the post and selected platform as a draft using React state.
8. Displayed all saved drafts dynamically using the map() function.
9. Cleared the text area after successfully saving the draft.
10. Tested the application by creating and saving posts for different platforms.

6. CODE

```
import React, { useState } from 'react';
function App() {
  const [platform, setPlatform] = useState('Instagram');
  const [post, setPost] = useState("");
  const [image, setImage] = useState(null);
  const [drafts, setDrafts] = useState([]);

  const maxChars = 2200;
  const handleSaveDraft = () => {
    if (!post.trim()) {
      alert('Please write something before saving.');
```

```
    return; }
    const newDraft = { platform, post, image: image ? image.name : 'No image', };
    setDrafts([newDraft, ...drafts]); alert('Draft saved successfully!'); };
```

```
  const handlePublish = () => {
    if (!post.trim()) {
      alert('Please write something before publishing.');
```

```
    return;
  }

  alert(`Post published on ${platform}!`);
  setPost("");
  setImage(null);
};
return (
  <div style={{ maxWidth: '900px', margin: '20px auto', padding: '20px', background: '#1f2b50',
    color: 'white', borderRadius: '12px', fontFamily: 'Arial' }}>
    <h1>📱 Social Media Composer</h1>
    <p>Create beautiful social media posts</p><label><b>Select Platform</b></label><br /><br />
    <select value={platform}
      onChange={(e) => setPlatform(e.target.value)} style={{
        width: '100%', padding: '12px', borderRadius: '8px', fontSize: '16px' }}>
      <option>Instagram</option>
```

```

<option>Facebook</option>
<option>LinkedIn</option>
<option>X</option>
</select><br /><br /><textarea rows='8' value={post}
onChange={(e) => setPost(e.target.value)}
placeholder='Write your post here...' style={{ width: '100%', padding: '15px', borderRadius: '10px',
fontSize: '18px', background: '#10214f', color: 'white', border: 'none' }} /><br /><br />
<div style={{ display: 'flex', alignItems: 'center' }}>
<input type='file' onChange={(e) => setImage(e.target.files[0])}/>
<div style={{ marginLeft: 'auto', color: '#66a3ff' }}>{post.length}/{maxChars}
</div></div><br />

```

```

<div style={{ display: 'flex', justifyContent: 'flex-end', gap: '10px' }}>
<button onClick={handleSaveDraft} style={{ background: '#16c784', color: 'white', border: 'none',
padding: '12px 20px', borderRadius: '8px', cursor: 'pointer' }} >

```

 Save Draft

```

</button><button
onClick={handlePublish} style={{ background: '#3b82f6',
color: 'white',
border: 'none',
padding: '12px 20px',
borderRadius: '8px',
cursor: 'pointer'
}}>

```

 Publish

```

</button></div><br />

```

```

<h2>Saved Drafts</h2>{drafts.length === 0 ? (
<p>No drafts available.</p>) : (
drafts.map((draft, index) => (<div
key={index}
style={{
background: '#10214f',
padding: '15px',
borderRadius: '10px',
marginBottom: '10px' }}>
<h3>{draft.platform}</h3>
<p>{draft.post}</p>
<small>Image: {draft.image}</small>
</div>)))}
</div>
);
}

```

```

export default App;

```

7. OUTPUT

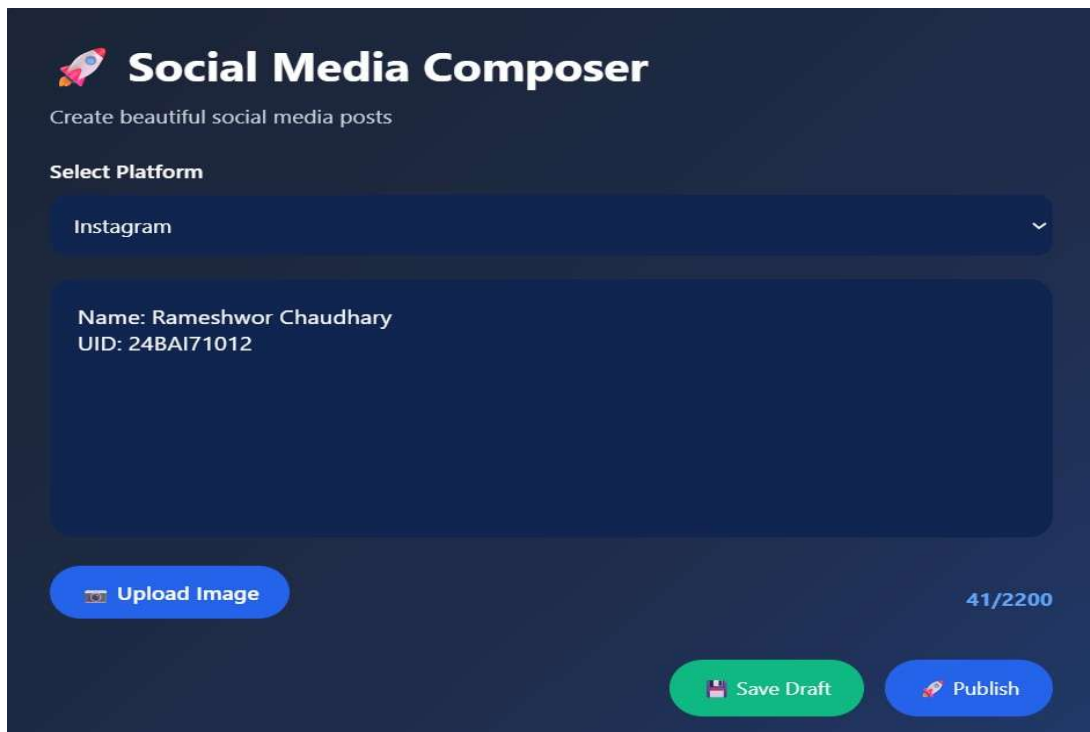
The image shows a web application titled "Social Media Composer" with a rocket icon. Below the title is the tagline "Create beautiful social media posts". A "Select Platform" dropdown menu is set to "Instagram". A text input area contains the name "Rameshwor Chaudhary" and the UID "24BAI71012". Below the text area is a character counter showing "41/2200". At the bottom, there are three buttons: "Upload Image" (with a camera icon), "Save Draft" (with a floppy disk icon), and "Publish" (with a rocket icon).

Fig 1.1: Social Media Post Composer User Interface

Explanation:

The output shows a Social Media Composer application developed using React.js. The application opens with a clean and attractive interface where the user can create posts for different social media platforms. In this output, Instagram is selected from the platform dropdown.

The user has entered "Rameshwor Chaudhary" and "UID: 24BAI71012" in the text area. The application also displays a character counter (41/2200), which shows the number of characters typed by the user and the maximum limit allowed. This helps users keep track of their post length.

The Upload Image button allows the user to attach an image with the post. The Save Draft button saves the current post so that it can be edited later, while the Publish button is used to publish the post to the selected platform (or simulate publishing in this project).

8. LEARNING OUTCOME

- Understand the core concept of the experiment.
- Apply theoretical knowledge to solve practical problems.
- Write, compile, and execute programs successfully.
- Analyze program output and identify logical errors.
- Improve programming, debugging, and problem-solving skills.

EXPERIMENT – 1.2

1. **AIM:** To implement a draft management system that allows users to save, retrieve, and manage post drafts within the frontend, with optional simulation of backend interactions.
2. **OBJECTIVE:**
 1. To manage draft data using React state.
 2. To implement Create, Read, Update, and Delete (CRUD) operations for drafts.
 3. To understand asynchronous workflows in frontend applications.
 4. To simulate backend communication using mock APIs or local storage.
3. **SOFTWARE REQUIREMENTS:**
 1. **Operating System:** Windows 10/11 or Linux
 2. **Programming Software/IDE:** Visual Studio Code /MySQL Workbench.
 3. **Processor:** Intel Core i3 or higher
 4. **RAM:** Minimum 4 GB

4. **THEORY**

A Draft Management System allows users to temporarily save content before publishing it. In React.js, this functionality is implemented using state management techniques such as `useState`, where draft data is stored and updated dynamically. Users can perform CRUD (Create, Read, Update, Delete) operations on drafts without requiring an actual backend server. For improved user experience, drafts may also be stored in the browser using `localStorage` or simulated with mock APIs to imitate asynchronous server communication. This approach helps developers build and test frontend features independently before integrating real backend services.

In this experiment, the Draft Management System is developed using React.js and the `useState` Hook. The application stores the selected platform, post content, uploaded image, and saved drafts in React state. Users can perform CRUD (Create, Read, Update, and Delete) operations on drafts without connecting to a real backend server. React automatically updates the user interface whenever the state changes, providing a smooth and interactive experience.

The application also supports image upload, editing existing drafts, and publishing posts. The Publish button simulates the publishing process by displaying a success message, while the Edit and Delete buttons allow users to modify or remove saved drafts. This experiment demonstrates important frontend concepts such as state management, event handling, conditional rendering, and dynamic UI updates, which are essential for building modern full-stack web applications. Proper state organization also makes applications easier to maintain and extend with backend integration in the future.

5. IMPLEMENTATION/PROCEDURE

1. Created a React application using Create React App.
2. Imported the **useState** Hook for state management.
3. Created state variables for platform, post, drafts, image, and editing mode.
4. Designed the user interface with dropdown, textarea, image upload, and action buttons.
5. Implemented Save Draft functionality to store posts dynamically.
6. Added Edit Draft functionality to modify existing drafts.
7. Implemented Delete Draft functionality to remove unwanted drafts.
8. Added Publish functionality to simulate post publishing.
9. Displayed all saved drafts dynamically using the map() function.
10. Tested all CRUD operations and verified that the application works correctly with dynamic state updates.

6. CODE

```
import React, { useState } from "react";
import "./App.css";

function App() {
  const [selectedPlatform, setSelectedPlatform] = useState("Instagram");
  const [postText, setPostText] = useState("");
  const [draftList, setDraftList] = useState([]);

  const [previewImage, setPreviewImage] = useState(null);
  const [editIndex, setEditIndex] = useState(-1);
  const handleSaveDraft = () => {
    if (postText.trim() === "") {
      alert("Please enter a post.");
      return;
    }

    const newDraft = {
      platform: selectedPlatform,
      content: postText,
      image: previewImage,
    };
    if (editIndex >= 0) {

      const updatedDrafts = [...draftList];
      updatedDrafts[editIndex] = newDraft;
      setDraftList(updatedDrafts);
      setEditIndex(-1);
    } else {
      setDraftList([newDraft, ...draftList]);
    }
    setPostText("");
    setPreviewImage(null);
  };

  const handleEdit = (index) => {
    const draft = draftList[index];
    setSelectedPlatform(draft.platform);
```

```

setPostText(draft.content);
setPreviewImage(draft.image);
setEditIndex(index);
};
const handleDelete = (index) => {
const remainingDrafts = draftList.filter((_, i) => i !== index);
setDraftList(remainingDrafts);
};
const handlePublish = () => {
alert("Post Published Successfully!");
setPostText("");
setPreviewImage(null);
};
const handleImageUpload = (e) => {
const file = e.target.files[0];
if (file) {
setPreviewImage(URL.createObjectURL(file));
}
};
return (
<div className="app">
<div className="card">
<h1>📝 Draft Management System</h1>
<select value={selectedPlatform}
onChange={(e) => setSelectedPlatform(e.target.value)}
>
<option>Instagram</option>
<option>Facebook</option>
<option>LinkedIn</option>
<option>X (Twitter)</option>
</select>
<textarea
placeholder="Write your post..."
value={postText}
onChange={(e) => setPostText(e.target.value)}
/>
{previewImage && (
<img
src={previewImage}
alt="Preview"
className="preview"
/>
)}
<input type="file" onChange={handleImageUpload} />

<button onClick={handleSaveDraft}>

```

```

    {editIndex >= 0 ? "Update Draft" : "Save Draft"
    }
  </button>

  <button onClick={handlePublish}>Publish</button>
</div>

<div className="draftSection">
  <h2>Saved Drafts</h2>

  {draftList.length === 0 ? (
    <p>No Drafts Available</p>
  ) : (
    draftList.map((draft, index) => (
      <div key={index} className="draftCard">
        <h3>{draft.platform}</h3>

        <p>{draft.content}</p>

        {draft.image && (
          <img
            src={draft.image}
            alt="Draft"
            width="150"
          />
        )}

        <button onClick={() => handleEdit(index)}>
          Edit
        </button>

        <button onClick={() => handleDelete(index)}>
          Delete
        </button>
      </div>
    ))
  )}
</div>
</div>
);
}

```

```
export default App;
```

7. OUTPUT

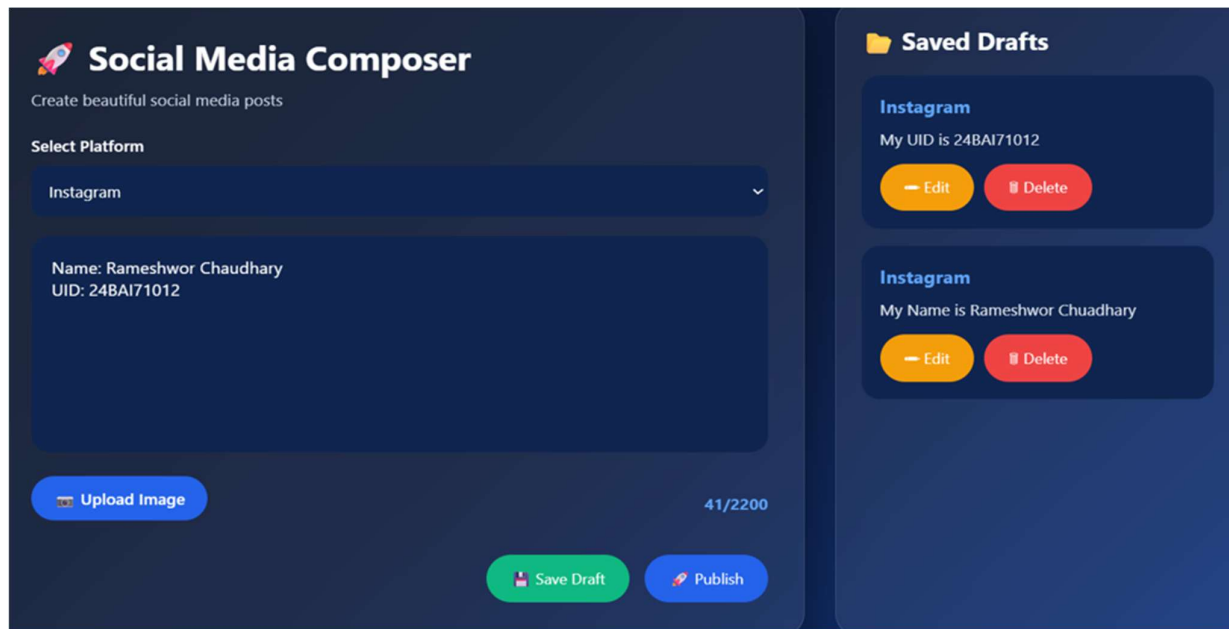


Fig 1.2: Draft Management System Webpage

Explanation:

The output displays a Draft Management System developed using React.js. The user can select a social media platform, write a post, upload an image, and save it as a draft. The interface is simple and user-friendly, making it easy to create and manage posts.

The Saved Drafts section on the right shows all the drafts created by the user. Each draft includes the selected platform and post content along with Edit and Delete buttons. The Edit button allows users to modify an existing draft, while the Delete button removes it from the list. This demonstrates the successful implementation of CRUD (Create, Read, Update, Delete) operations and React state management.

8. LEARNING OUTCOME

- Understand React state management using the useState Hook for handling dynamic data.
- Implement CRUD (Create, Read, Update, Delete) operations to manage post drafts efficiently.
- Develop a Draft Management System for creating, editing, deleting, and viewing saved drafts.
- Handle user interactions such as image upload, platform selection, and post publishing in a React application.
- Apply event handling, conditional rendering, and dynamic UI updates to build an interactive frontend.
- Improve programming, debugging, and problem-solving skills while developing a responsive React application.